

NDNSF-UAV-APP

**Named Services for Secure Control, Adaptive Video, and Multi-UAV
Collaboration**

Tianxing Ma

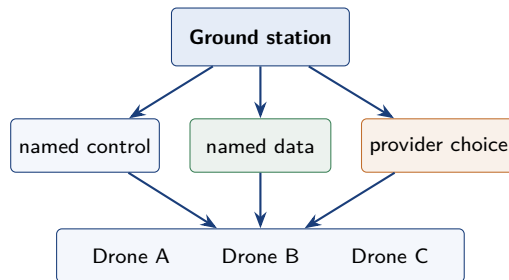
Department of Computer Science
University of Memphis

August 2026

Why an NDN-Native UAV Application?

One mission combines different communication needs

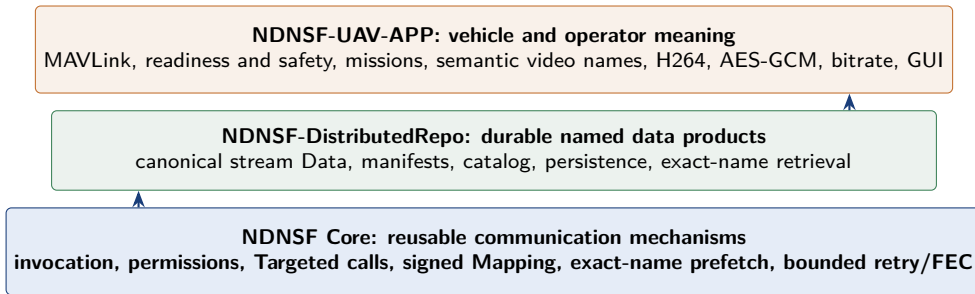
- ▶ Safety-sensitive commands need authorization and prompt failure.
- ▶ Telemetry and readiness must stay tied to the selected vehicle.
- ▶ Live video and retained playback share canonical Data; exported files and logs are durable objects.
- ▶ Multiple mobile drones may be busy, disconnected, or replaceable.



The application binds UAV operations to **service and data names**, not to one fixed endpoint connection.

Evidence: [NDNSF-UAV-APP/README.md](#): Why This Application Exists

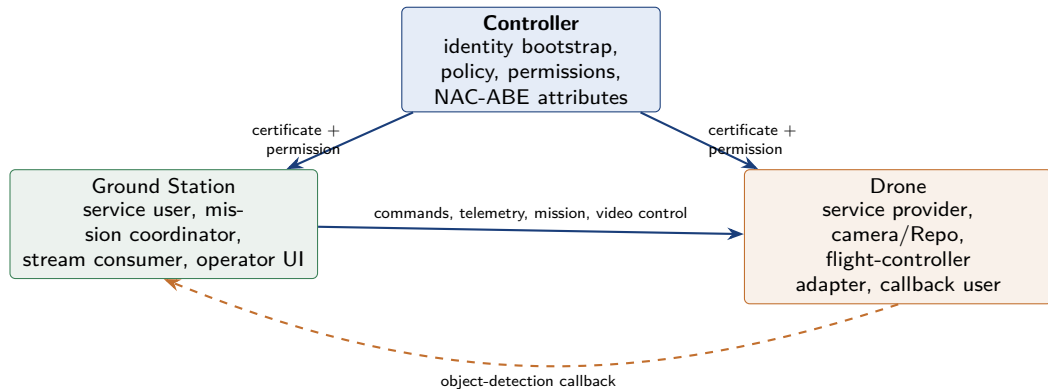
Responsibility Boundary



Core transports opaque bytes under application names; UAV owns video meaning and encryption, while Core owns the reusable live network loop.

Evidence: docs/ndnsf-core-app-boundary.md; Spec 118/119 ownership contracts

Three Runtime Roles

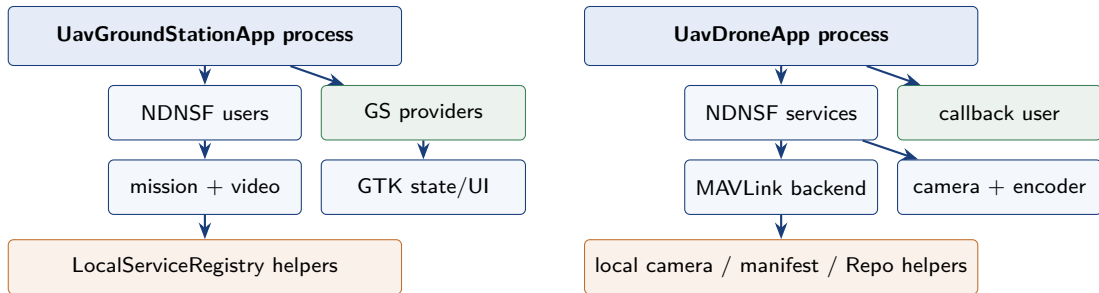


Typical PC: Controller + Ground Station + NFD

Each vehicle: DroneAPP + NFD + camera + MAVLink link

Evidence: UavDroneApp.cpp; UavGroundStationApp.cpp; README deployment section

Service Containers, Not One Monolithic RPC



One process shares identity, configuration, hardware, and lifecycle; every remote service remains separately named and permission-controlled.

Evidence: [DroneServiceContainer.inc.hpp](#); [GroundStationServiceContainer.inc.hpp](#)

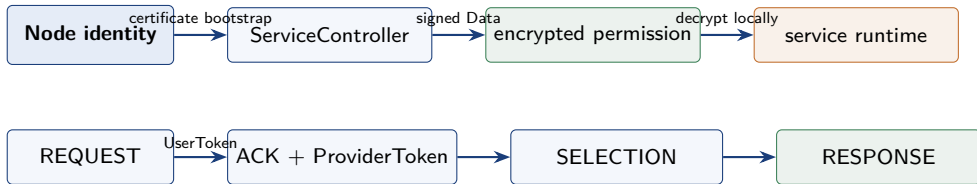
Named Services and Invocation Modes

Name	Mode	Purpose
/UAV/MAVLink/Execute	Targeted only	known-drone low-latency command path
/UAV/Telemetry/GetStatus	normal + Targeted	discovery or selected-drone state refresh
/UAV/Mission/Assign	normal	multi-provider mission-slot selection
/<drone>/UAV/Camera/Video	normal	provider-specific Start/Stop control
/<drone>/UAV/MAVLink/Parameters	normal	vehicle capability and parameter snapshot
/<drone>/UAV/Preflight/Checklist	normal	typed operator readiness details
/<drone>/UAV/Camera/Recording/Manifest	normal	discover retained canonical stream Data
/UAV/GS/ObjectDetection	normal	drone-to-ground-station compute callback

Shared service names allow provider selection; provider-specific names address one physical vehicle or its data product.

Evidence: `shared/UavNames.hpp`; `DroneServiceContainer::installServiceInstances`

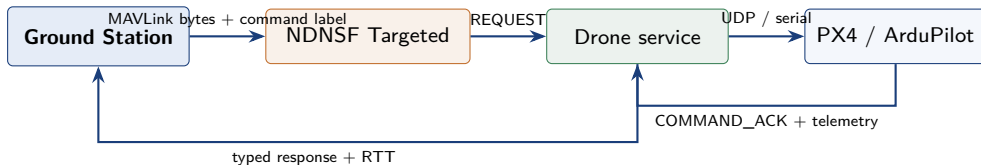
Secure Startup and Invocation



- ▶ REQUEST/SELECTION use service attributes; ACK/RESPONSE use permission attributes.
- ▶ One-time user/provider tokens bind the handshake and reject replay.
- ▶ Controller permission Data is signed and encrypted to the target identity certificate.

Evidence: ServiceController; ServiceUser/ServiceProvider security path; UAV deployment README

Targeted MAVLink Command Path



First use

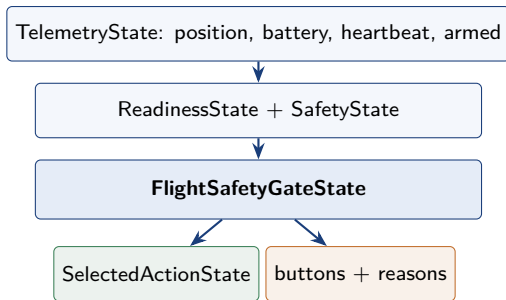
- ▶ authenticated ACK/selection bootstrap
- ▶ obtain a batch of one-time token pairs

Later commands

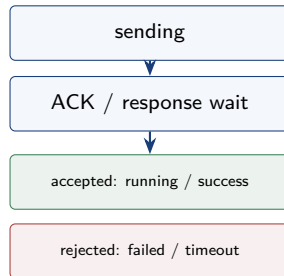
- ▶ request/response-only Targeted calls
- ▶ permissions, token checks, and replay protection remain

Evidence: `sendMavlinkCommandToDrone`; `ServiceProvider TargetedOnly` registration

Typed State Drives the Operator Workflow



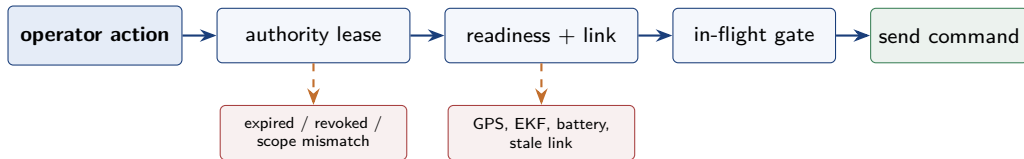
Command lifecycle



The GUI reads typed state. Stale or lost telemetry changes the same safety model used by the actual command-send path.

Evidence: `shared/UavProtocol.hpp`; `GroundStationRuntimeState.hpp`; `commandLifecycle`

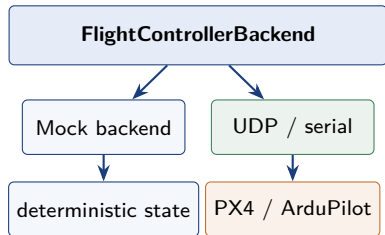
Safety Gates and Operator Authority



- ▶ A lease binds operator, drone set, monitor/control scope, and time window.
- ▶ Monitor authority permits observation but does not permit flight control.
- ▶ Authority never overrides readiness or safety; both gates must pass.
- ▶ Rejection reasons and authority alerts remain visible and auditable.

Evidence: `OperatorAuthorityLease`; `validateOperatorLease`; `validateFlightSafetyGate`

Flight-Controller Adapter Boundary



Backend contract

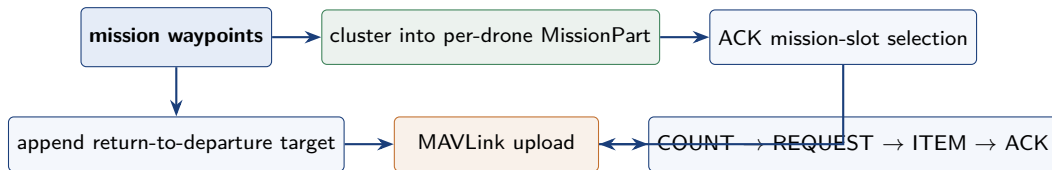
- ▶ forward MAVLink frames
- ▶ return command ACK fields
- ▶ expose latest telemetry
- ▶ read/edit parameters
- ▶ upload mission waypoints

Real-link helpers

- ▶ GCS heartbeat
- ▶ manual-control replay
- ▶ neutral command after timeout

Evidence: FlightControllerBackend; MockFlightControllerBackend; UdpFlightControllerBackend

Multi-Drone Mission Lifecycle

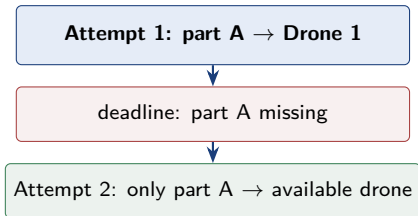


Mission start is phased: arm accepted drones, take off accepted drones, then start only the mission parts that were uploaded successfully.

Evidence: MissionPlan/MissionPart; uploadMissionPlan; executeMissionWaypoints

Mission Recovery and Persistence

Application-level compensation



- ▶ shared `patrol_task_id`; fresh NDNFSF request each attempt
- ▶ no restart of already completed mission parts

Persistent mission document

- ▶ stable plan and operator identifiers
- ▶ per-drone parts and waypoint order
- ▶ geofence/rally metadata for future expansion
- ▶ save, load, preview, and upload through one typed model

Compensation repairs missing service results; it is not autonomous in-flight route optimization.

Evidence: `MissionProgressState`; `MissionPlanDocument`; `runPatrolCompensationTask`

Setup and Inspection Services

Parameters

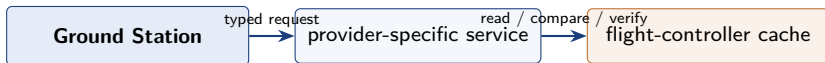
- ▶ firmware, modes, completeness
- ▶ typed value and dry-run
- ▶ expected-value check
- ▶ read-back verification

Preflight

- ▶ named checklist items
- ▶ severity and blocking state
- ▶ operator-readable reason
- ▶ refresh before dangerous action

Analyze

- ▶ MAVLink message summaries
- ▶ active rates and last-seen state
- ▶ vehicle/mission snapshot
- ▶ inspector-oriented output



Evidence: VehicleParameterEditRequest/Result; PreflightCheckItem; UavAnalyzeSnapshot

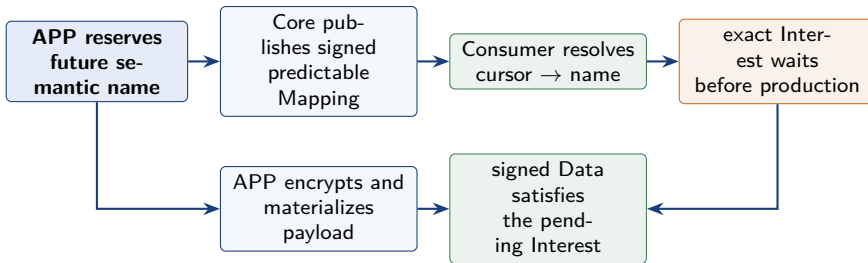
Live Video: Service Control Opens an Independent Stream



- ▶ Drone uses `createLiveStream`; Ground Station uses `openLiveStream`.
- ▶ The response authorizes one Stream and returns its descriptor/key; video remains a separate named-data path.
- ▶ Application lifecycle owns start/stop; Core stop is idempotent and suppresses retired-session callbacks.

Evidence: `ServiceProvider::createLiveStream`; `ServiceUser::openLiveStream`; Spec 118 camera-start contract

How Prefetch Preserves Meaningful Data Names

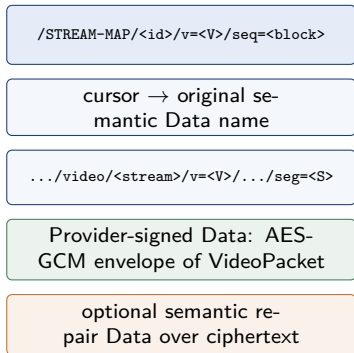


- ▶ Only Mapping names are sequential and constructible; cursor is an internal ordering key.
- ▶ Payload Data keeps its immutable application name; Mapping carries names, never nested payload Data.
- ▶ Late Mapping earns no prefetch credit.

Predict Mapping, then issue the real semantic-name Interest early: no selector discovery or Data-in-Data encapsulation.

Evidence: StreamNameMapBlock/Resolver; LiveStreamPublisher::reserveAhead; Spec 119 FR-001/FR-026

Semantic Name, Encryption, and Session Model



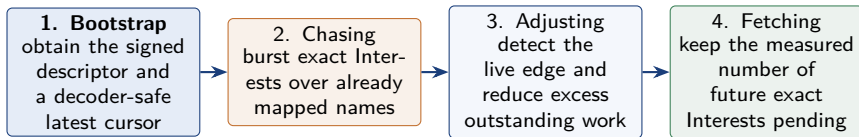
Consumer guards

- ▶ validate descriptor, Mapping chain, exact name, and expected Provider
- ▶ authenticate AES-GCM name/session/cursor binding before decode
- ▶ reject replay, stale session, wrong signer, tag, digest, or nonce
- ▶ accept recovered opaque bytes only through the same AEAD gate

Core never receives the key or plaintext; the cursor never replaces the payload's NDN name.

Evidence: UavVideoAad; HybridMessageEnvelope; StreamNameMapBlock; Spec 118

Paper-Guided Prefetch: Four Receiver Phases



Paper's demand estimate

$$D_{sample} = \left\lceil \frac{DRD}{T} \right\rceil$$

- ▶ *DRD*: exact Interest expression to matching Data reception
- ▶ *T*: measured sample production period

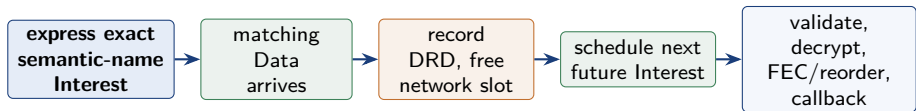
NDNSF translation

$$D_{packet} = D_{sample} \times itemsPerSample + reserve$$

- ▶ phase window controls the real cursor horizon
- ▶ segmented/FEC media keeps one bounded source-group reserve

Evidence: Gusev et al. 2016; LiveStreamPrefetchController; StreamFetchDecision; Spec 123

The Interest Pipeline Stays Full Before APP Processing



Two independent bounded owners

- ▶ network in-flight: consumes Interest capacity
- ▶ payload processing: owns received cursors until APP completion
- ▶ both sets reject duplicates and drain on stop

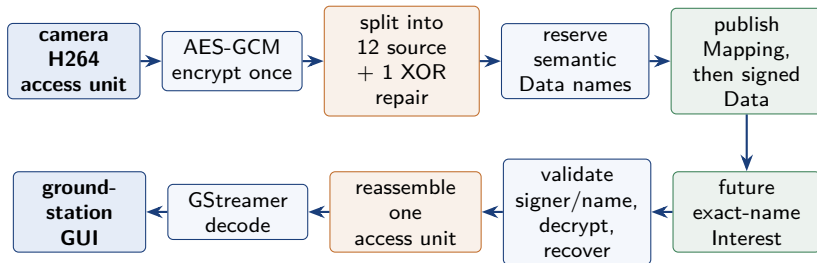
Why this matters

- ▶ signature validation, decryption, decoding, and GUI work cannot stall Interest refill
- ▶ a pending exact Interest lets newly produced Data return immediately—approximately one-way network delay, not an extra RTT

Reception refills the network pipeline before APP processing. Prefetch succeeds only when the Provider sees the exact Interest before materializing that Data.

Evidence: `LiveStreamConsumerHandle::schedule`; `m_payloadProcessing`; payload expression timestamps; Spec 123

How UAV Video Uses the Generic NDN SF Stream



- ▶ UAV APP owns H264, 3600-byte packetization, AES-GCM keys/AAD, 12+1 FEC, frame assembly, decoder, and GUI.
- ▶ NDN SF Core owns signed 32-name Mapping blocks, cursor resolution, exact-name Interests, DRD, phase/window control, bounded retry, ordering, and opaque-byte recovery.
- ▶ The application Data name remains semantic and Provider-signed; Mapping contains only cursor-to-name bindings, never video payload.

Evidence: UavGroundStationApp; UavVideoPipeline; StreamNameMapBlock; LiveStreamPublisher/ConsumerHandle

What Was Wrong—and What Changed

Problem	Old behavior	Corrected behavior
Delay feedback Pipeline refill Phase control	capture age treated as network RTT waited for validation/decrypt/APP callback phase/window changed diagnostics only	exact Interest-to-Data DRD only refill immediately on Data reception Chasing/Adjusting window changes actual issued cursors
Media demand Publisher lifecycle	one extra packet reserved evicted Data could look “not produced”	one complete 12-packet source group reserved materialized history distinguishes evicted from future
Mapping work Measurement	16 names per signed block per-packet DEBUG trace perturbed latency	32 names per block, still about 4.3 KiB sampled frame trace retained; packet trace independently disabled

This historical Spec 123 failure was prefetch underfill. It is distinct from the later Spec 156 inverse FPS/latency trend, which the next stage-level diagnostic localizes before network fetch/reassembly.

Evidence: Stream.cpp; TimelineTrace.cpp; Spec 123 evidence sequence

Corrected 60-Second MiniNDN Result

Original UAV workload	Measured
Video / packetization	8 Mbps, 3600 B, 12+1
Decoded frames	1,830
Exact sampled identities	174/174 (100%)
Decoder startup	18 ms
Capture → decode p50	142.654 ms
Capture → decode p95	205.946 ms
Capture → decode p99	269.402 ms
Decode → widget p95	4.076 ms

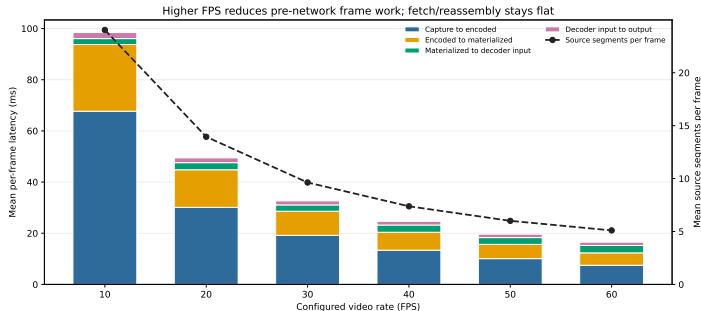
Network and safety evidence

- ▶ Provider future hits: 7,153/7,153
- ▶ Mapping + payload work: 13.435 Interests/frame
- ▶ only 3.35% above the configured 13 stream items/frame
- ▶ FEC recovered 11 opaque items
- ▶ rejected correlations: 0
- ▶ security failures: 0
- ▶ pending/in-flight at stop: 0/0

The original high-packet-rate UAV stream now stays near the live edge; the earlier 1,052 ms result is superseded for this corrected zero-loss configuration.

Evidence: [results/spec123-paper-prefetch-sample-reserve-20260719-060053](#); Spec 123 completion summary

Why Higher FPS Shows Lower Latency



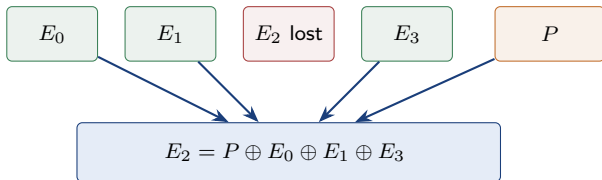
Strict per-frame replay

- ▶ 15,969/15,969 exact timelines; 100% coverage
- ▶ 10→60 FPS mean end-to-end reduction: 81.923 ms
- ▶ capture/encode + materialization explain 99.392% of that reduction
- ▶ source segments/frame: 24.045→5.103
- ▶ materialized→decoder input: 2.320→2.884 ms

The inverse FPS/latency trend is not evidence of insufficient future Interests: fetch/reassembly does not improve. At fixed bitrate, higher FPS produces smaller frames and less pre-network work per frame.

Evidence: Spec 157; immutable offline replay of the six Spec 156 cells; no MiniNDN rerun

Optional XOR Recovery over Opaque Ciphertext



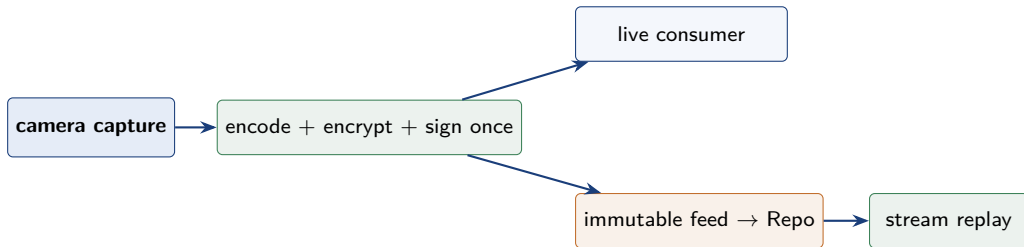
Contract v1

- ▶ disabled by default; at most one XOR repair item
- ▶ Core sees encrypted opaque bytes, not H264 plaintext
- ▶ signed metadata binds Provider, names, lengths, and digests
- ▶ recovered bytes are AEAD-checked and never cached as original Data

One-loss recovery is deterministically correct, but current MiniNDN runs showed no recovery event or performance gain.

Evidence: LiveStreamFecOptions; publishGroup; FecRecovered provenance; Spec 118/119 evidence

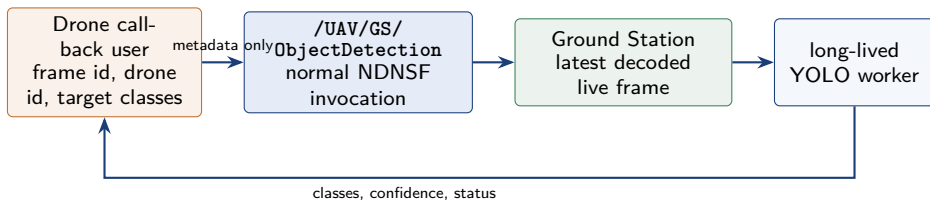
One Canonical Video Stream, Two Uses



- ▶ Viewer and retention lifecycles are independent; the media representation is not.
- ▶ Repo stores the original semantic name, ciphertext, signature, and wire digest unchanged.
- ▶ Historical playback reuses normal LiveStream validation, decryption, ordering, and decoding.

Evidence: Core PublishedPacketFeed; CanonicalVideoRecordingManifest; Spec 120 acceptance evidence

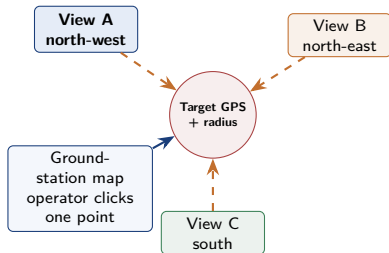
Ground-Station Object Detection Callback



Roles can reverse: a drone is normally a provider, but it can consume a heavier service from the ground station. Image bytes are reused locally and are not inserted into the service request.

Evidence: `installObjectDetectionService; startObjectDetectionLoop; yolo_detect_worker.py`

Operator-Initiated Multi-UAV Geo-Capture



Concrete task

1. The operator marks a fixed ground location on a map or georeferenced image.
2. A protected GeoCaptureRequest discovers reachable UAVs; after ACK closure, the plan assigns complementary view sectors and capture slots.
3. Selected UAVs publish signed images; an Assembler publishes one MultiViewImageSet manifest.

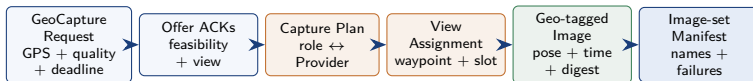
Deliberate MVP boundary

The GPS point identifies a place, not a moving object's identity. No cross-camera Re-ID or target triangulation is required.

Request Data defines the GPS point, time, view, and quality requirements; ACK Data offers pose quality, ETA, battery, sensor capability, and willingness. The closed ACK set drives the plan.

Evidence: Generic collaboration API in ServiceUser/ServiceProvider; MAVLink ROI and camera-capture messages

The Plan Becomes Named Geo-Capture Policy



Data-plane acceptance checks

- ▶ current request/plan/attempt, assigned role, selected Provider signature, and authorized name/key scope
- ▶ capture inside the assigned slot and region, with frame-bound time, pose, gimbal attitude, calibration ID, and digest
- ▶ explicit failure Data for unsafe flight, occlusion, or missed quality threshold

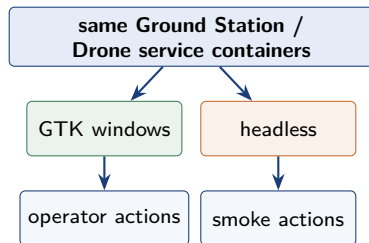
Responsibility boundary

- ▶ NDNFSF owns discovery, ACK closure, role assignment, named dependencies, authorization, and assembly.
- ▶ The UAV application owns viewpoint generation, path planning, gimbal aim, and image-quality tests.
- ▶ The autopilot retains final authority over geofence, collision, readiness, and safety rejection.

The result is a verifiable image set for one geographic point; recognition or 3-D reconstruction is a separate downstream service.

Evidence: BeginCollaboration → ACK_CLOSED → CommitCollaborationPlan; MAV_CMD_DO_SET_ROI_LOCATION and CAMERA_IMAGE_CAPTURED

One Runtime, GUI or Headless



Evidence: MiniNDN UAV launcher; UAV protocol/state unit tests

Validation layers

- ▶ C++ protocol/state unit tests
- ▶ deployment and config quick smoke
- ▶ MiniNDN multi-node service tests
- ▶ PX4 SITL + jMAVSim command/mission tests
- ▶ GUI smoke under Xvfb

Default MiniNDN placement

- ▶ Controller + GS: Memphis
- ▶ Drone A: UCLA
- ▶ Drone B: WUSTL

Implemented Foundation and Current Boundary

Implemented foundation

- ▶ named, permission-controlled UAV services
- ▶ Targeted command path and typed control state
- ▶ operator authority and safety gates
- ▶ multi-drone mission assignment and compensation
- ▶ semantic-name LiveStream: AES-GCM, signed Mapping, bounded prefetch, optional XOR
- ▶ encrypted recording discovery and playback

Current boundary and work in progress

- ▶ map-selected GeoCaptureRequest
- ▶ ACK-derived viewpoint/Assembler role planning
- ▶ CaptureOffer, ViewAssignment, GeoTaggedImage, and MultiViewImageSet schemas
- ▶ safety-valid capture execution and named result assembly
- ▶ end-to-end completion, view diversity, latency, bytes, mobility, and loss evaluation
- ▶ production flight-safety assurance remains out of scope

Next: implement map-triggered multi-UAV geo-capture, then measure valid-view completion, angular diversity, latency, communication cost, and resilience to mobility and Data loss.

Evidence: NDNSF-UAV-APP/README.md; Spec 118/119 contracts; generic collaboration API; proposed UAV workflow